



UNIVERSIDAD DE MÁLAGA

Grado en Ingeniería Electrónica, Robótica y Mecatrónica

Memoria de Práctica

Lab-MR 3

Navegación reactiva con campos potenciales

Navegación mediante campos potenciales artificiales

Ampliación de Robótica

Saúl Gutiérrez Rodríguez

Pablo Haro García

Mario Merino Prado

Repositorio: github.com/marioomerino24/ampliacion_robotica

Entorno: MATLAB

Índice

1. Introducción	3
1.1. Objetivo de la práctica	3
1.2. Descripción del problema	3
2. Fundamentos teóricos	3
2.1. Modelo de planta y sensor	3
2.2. Campo atractivo	4
2.3. Campo repulsivo	4
2.4. Ley de movimiento	4
3. Diseño e implementación	5
3.1. Arquitectura de la solución	5
3.2. Parámetros utilizados	5
3.3. Procesamiento sensorial	5
3.4. Implementación del bucle de control	6
3.5. Detección de mínimos locales	6
4. Resultados experimentales	6
4.1. Escenario de prueba	6
4.2. Análisis cualitativo	7
4.3. Sensibilidad a los parámetros	7
4.4. ¿Pueden los parámetros resolver un mínimo local?	8
5. Discusión	8
5.1. Problemas encontrados y soluciones	8
5.2. Limitaciones del método	9
6. Conclusiones	9

1. Introducción

1.1. Objetivo de la práctica

El objetivo de esta práctica es implementar una estrategia de **navegación reactiva** basada en **campos potenciales artificiales**. A partir de un mapa de ocupación binario y de un láser de barrido simulado, el robot debe alcanzar un destino seleccionado por el usuario evitando los obstáculos detectados en cada iteración, sin disponer de un plan global previo.

1.2. Descripción del problema

Dado un mapa de ocupación y dos puntos q_{ini} (origen) y q_{dest} (destino) seleccionados sobre dicho mapa, el robot debe construir en cada paso una dirección de avance que combine:

1. una **atracción** hacia el destino,
2. una **repulsión** ejercida por los obstáculos más cercanos.

La trayectoria se construye paso a paso, sin almacenar memoria del mapa global, lo que la convierte en un método puramente **reactivo**. La eficacia depende críticamente de los coeficientes α y β y del rango de influencia D , y el método es susceptible a quedar atrapado en **mínimos locales**, que se detectan por límite de iteraciones.

2. Fundamentos teóricos

2.1. Modelo de planta y sensor

Se considera un robot puntual con pose $q = (x, y, \theta)$ que se desplaza un paso fijo v en la dirección indicada por la fuerza resultante. Como sensor, se simula un **láser de barrido** mediante `rayIntersection` sobre el mapa de ocupación, con resolución 1° en el rango $[-\pi/2, \pi/2]$ y un alcance máximo $r_{max} = 10$ m.

2.2. Campo atractivo

La componente atractiva tira del robot hacia el destino. Se utiliza una formulación unitaria normalizada por la distancia al objetivo:

$$\mathbf{F}_{\text{atr}} = \alpha \frac{q_{\text{dest}} - q}{\|q_{\text{dest}} - q\|} \quad (1)$$

con $\alpha > 0$ el coeficiente de atracción. La dirección de $\mathbf{F}_{\text{atr}}$ apunta siempre al destino y su magnitud es constante e igual a α .

2.3. Campo repulsivo

Cada obstáculo $q_{\text{obs},i}$ situado a distancia $d_i = \|q - q_{\text{obs},i}\|$ ejerce sobre el robot una fuerza repulsiva si $d_i < D$:

$$\mathbf{F}_{\text{rep},i} = \beta \left(\frac{1}{d_i} - \frac{1}{D} \right) \frac{1}{d_i^2} \frac{q - q_{\text{obs},i}}{\|q - q_{\text{obs},i}\|} \quad (2)$$

Esta expresión cumple las propiedades exigidas:

- es **infinita** cuando $d_i \rightarrow 0$ (frontera del obstáculo),
- se anula **exactamente** cuando $d_i = D$ (frontera de influencia),
- es siempre **repulsiva** (vector unitario desde el obstáculo hacia el robot).

La fuerza repulsiva total se obtiene sumando sobre todos los obstáculos dentro del rango de influencia:

$$\mathbf{F}_{\text{rep}} = \sum_{i: d_i < D} \mathbf{F}_{\text{rep},i} \quad (3)$$

2.4. Ley de movimiento

La fuerza resultante $\mathbf{F}_{\text{res}} = \mathbf{F}_{\text{atr}} + \mathbf{F}_{\text{rep}}$ define la dirección de avance:

$$\theta = \text{atan2}(F_{\text{res},y}, F_{\text{res},x}) \quad (4)$$

y el robot ejecuta un paso de longitud v en esa dirección:

$$q_{k+1} = \begin{bmatrix} x_k + v \cos \theta \\ y_k + v \sin \theta \\ \theta \end{bmatrix} \quad (5)$$

El bucle se repite hasta que $\|q_{\text{dest}} - q\| < v$ (destino alcanzado) o hasta superar un límite de 1000 iteraciones, en cuyo caso se asume que el robot ha quedado atrapado en un **mínimo local**.

3. Diseño e implementación

3.1. Arquitectura de la solución

La solución es un único script de MATLAB (`plantilla_campos_potenciales.m`) organizado en cuatro bloques:

1. Carga del mapa y selección interactiva de origen y destino.
2. Definición de parámetros del vehículo y del sensor.
3. Bucle de control: lectura del láser, cálculo de fuerzas, actualización de pose y dibujado incremental.
4. Verificación final: destino alcanzado o mínimo local.

3.2. Parámetros utilizados

La Tabla 1 resume los valores empleados, todos ellos constantes durante la ejecución.

Tabla 1: Parámetros del controlador de campos potenciales.

Parámetro	Valor	Descripción
v	0.4	Paso lineal del robot por iteración.
D	1.5	Rango de influencia del campo repulsivo.
α	1	Coeficiente de atracción.
β	100	Coeficiente de repulsión.
$r_{\max}$	10	Alcance máximo del láser.
$\Delta\alpha_{\text{laser}}$	1°	Resolución angular del barrido.
rango angular	$[-\pi/2, \pi/2]$	Sector explorado por el láser.
iter. máx.	1000	Límite para detectar mínimos locales.

3.3. Procesamiento sensorial

En cada iteración se llama a `SimulaLidar`, que envuelve a `rayIntersection` y devuelve, para cada ángulo, las coordenadas absolutas $(x_{\text{obs}}, y_{\text{obs}})$ del primer impacto o NaN si el rayo no intercepta nada dentro del alcance. El procesamiento aplica dos filtros sobre las salidas del sensor:

1. **Eliminación de NaN:** los rayos sin impacto se descartan mediante una máscara lógica.
2. **Recorte por rango:** solo se mantienen los obstáculos a distancia $d_i < D$, ya que la ecuación (2) se anula fuera de ese rango.

Este preprocesado es fundamental: si no se filtran los NaN la suma vectorial de la repulsión se contamina, y si no se acota por D se incluyen contribuciones nulas que aumentan el coste por iteración sin añadir información.

3.4. Implementación del bucle de control

El bucle del script implementa fielmente las ecuaciones (1)–(5). Las operaciones clave son:

- Cálculo vectorial de las distancias d_i a todos los rayos en una sola operación.
- Construcción de F_{atr} con un único vector unitario.
- Acumulación de F_{rep} mediante un bucle sobre los obstáculos cercanos.
- Actualización de la pose y dibujado incremental con `plot` y `drawnow`, lo que permite visualizar la trayectoria en tiempo real.

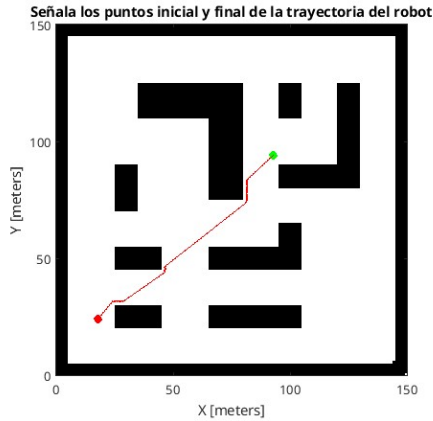
3.5. Detección de mínimos locales

La detección se realiza por simple recuento de iteraciones: si el bucle alcanza 1000 pasos sin satisfacer la condición de llegada se considera que el robot ha quedado bloqueado y se imprime el mensaje correspondiente. No se introducen mecanismos correctores (perturbación aleatoria, navegación tangencial, etc.); la mejora de este aspecto se aborda en la práctica integradora Lab-MR 6.

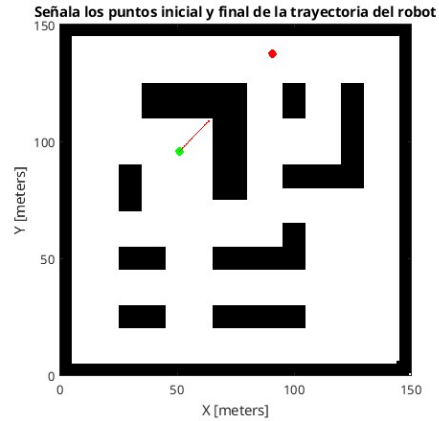
4. Resultados experimentales

4.1. Escenario de prueba

Las pruebas se han realizado sobre el mapa `mapa1_150.png`, un entorno cerrado con varios obstáculos rectangulares. El usuario selecciona origen y destino con `ginput`, y la trayectoria recorrida se dibuja en rojo sobre el mapa. La Figura 1 ilustra los dos comportamientos típicos del método: un caso en el que el robot alcanza el destino y otro en el que queda atrapado en un mínimo local.



(a) Caso favorable: el robot bordea los obstáculos y alcanza el destino con una trayectoria suave.



(b) Mínimo local: atracción y repulsión se cancelan y el robot queda atrapado tras agotar las 1000 iteraciones.

Figura 1: Trayectorias reactivas obtenidas con campos potenciales sobre el mapa `mapa1_150.png`. Círculo verde: origen. Círculo rojo: destino. Trazo roja: recorrido del robot.

4.2. Análisis cualitativo

Con los parámetros por defecto ($\alpha = 1$, $\beta = 100$, $D = 1.5$):

- En configuraciones **abiertas**, el robot avanza directamente hacia el destino con una desviación mínima al pasar cerca de paredes.
- En configuraciones **con obstáculos intermedios**, el robot bordea el obstáculo siguiendo una trayectoria suave (Figura 1a).
- En configuraciones con **obstáculos en forma de U** o cuellos estrechos enfrentados al destino, el robot puede quedar atrapado: la atracción hacia el destino y la repulsión combinada se cancelan, generando un **mínimo local** (Figura 1b).

4.3. Sensibilidad a los parámetros

La relación β/α controla el equilibrio entre evitación y avance:

- Con β demasiado pequeño, el robot rasca paredes o llega a colisionar.
- Con β demasiado grande, la trayectoria se aleja innecesariamente de los obstáculos y aumentan las oscilaciones.
- El valor $\beta = 100$ ofrece un compromiso adecuado en este mapa con $\alpha = 1$ y

$$D = 1.5.$$

El rango de influencia D actúa como filtro: incrementarlo hace que el robot empiece a desviarse antes, mientras que reducirlo produce reacciones bruscas en las cercanías. El valor $D = 1.5$ corresponde aproximadamente a tres pasos del robot, lo que da margen de reacción sin sobre-anticipación.

4.4. ¿Pueden los parámetros resolver un mínimo local?

El enunciado plantea explícitamente la siguiente pregunta: una vez situado el robot en una configuración de mínimo local, **¿puede el método encontrar la solución sólo modificando α , β o D ?**

La respuesta es **no**. Un mínimo local del campo potencial es un punto q^* donde $F_{\text{atr}}(q^*) + F_{\text{rep}}(q^*) = 0$. Modificar α y β **escala** ambas componentes pero **no cambia la geometría** del campo: si las fuerzas se cancelan en q^* con una pareja (α, β) , también se cancelan con cualquier $(\lambda\alpha, \lambda\beta)$, y reescalar solo una de ellas como (α', β') desplaza el punto de cancelación a otra ubicación del entorno pero **no elimina el mínimo**, simplemente lo translada. Modificar D tampoco rompe el mínimo: cambia qué obstáculos contribuyen a la repulsión, pero la cancelación vectorial se mantiene posible en cualquier configuración con simetría (cuello en U, paredes enfrentadas, etc.).

Por tanto, los mínimos locales son una limitación **estructural** del método, no un defecto de calibración. Su mitigación requiere extender el algoritmo, por ejemplo con perturbación aleatoria, navegación tangencial o información global. La práctica integradora Lab-MR 6 aborda esta cuestión añadiendo un campo tangencial y una maniobra de escape activada por detección de estancamiento.

5. Discusión

5.1. Problemas encontrados y soluciones

Problema 1: rayos sin impacto contaminan la repulsión. `rayIntersection` devuelve NaN cuando no hay intersección dentro del alcance, lo que invalida cualquier operación vectorial posterior. **Solución:** máscara lógica que descarta los rayos no válidos antes de calcular distancias y vectores repulsivos.

Problema 2: contribuciones repulsivas despreciables. Aunque la fórmula (2) se anula por construcción en $d_i = D$, evaluarla para todos los rayos del láser es ineficiente. **Solución:** filtrar previamente por $d_i < D$ y solo iterar sobre obstáculos

relevantes, con un coste despreciable en el resto.

Problema 3: mínimos locales. En zonas donde la atracción y la repulsión se cancelan exactamente, el robot oscila o se detiene. **Solución:** en esta práctica solo se **detectan** (límite de 1000 iteraciones); la mitigación se aborda en Lab-MR 6 mediante un campo tangencial y una maniobra de escape por estancamiento.

5.2. Limitaciones del método

- **Susceptibilidad a mínimos locales:** no se puede resolver únicamente ajustando α y β .
- **Ausencia de memoria global:** el robot no recuerda zonas exploradas, por lo que puede volver a entrar en la misma trampa.
- **Movimiento de paso fijo:** el avance constante v no se adapta a la cercanía de obstáculos, lo que puede provocar pasos demasiado grandes en zonas estrechas.
- **Modelo puntual del robot:** no se considera la geometría del vehículo; en escenarios reales habría que inflar los obstáculos.
- **Sensor frontal limitado a 180° :** los obstáculos por detrás no se ven, lo que es coherente con el método reactivo pero no permite reculadas seguras.

6. Conclusiones

Se ha implementado un controlador de **navegación reactiva por campos potenciales artificiales** que dirige al robot hacia un destino seleccionado evitando los obstáculos detectados por el láser. El controlador es simple, transparente y eficiente, y reproduce con fidelidad las propiedades teóricas del método: campo atractivo unitario hacia el destino, campo repulsivo que diverge en la frontera del obstáculo y se anula en la frontera de influencia, y combinación vectorial directa para definir la dirección de avance.

Las pruebas confirman que el método es **eficaz** en escenarios moderadamente complejos pero **insuficiente** en presencia de configuraciones que generan mínimos locales, donde la única salida es disponer de información global. Esta limitación motiva la práctica Lab-MR 6, en la que la planificación global (Dijkstra/A*) actúa como guía y los campos potenciales se reducen a la ejecución local entre subobjetivos, complementados con un campo tangencial y una maniobra de escape por estancamiento.